



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO



Tema:

Investigacion Unidad 1 de Programación en Ambiente Cliente/Servidor

Docente:

Jenner Noel Che Méndez

Carrera:

Ingeniería en informática

Alumnos:

José Abimael Cisneros Escamilla

Semestre:

7AI

Fecha:

4 de octubre de 2026

Unidad 1. Arquitecturas cliente-servidor contemporáneas y diseño API-first

Índice

Introducción3

1.1. Evolución del modelo cliente-servidor4

2.2. Fundamentos de red aplicados6

2.3. Arquitecturas de software contemporáneas.....9

2.4. Estilos y protocolos de comunicación 12

2.5. Separación de responsabilidades y criterios arquitectónicos 15

2.6. Diseño API-first, diagramas C4, contrato inicial y ADR 17

Conclusión..... 20

Fuentes de consulta — formato APA 7 21

Introducción

El desarrollo de sistemas de información ha evolucionado de aplicaciones aisladas ejecutadas en una sola computadora hacia sistemas distribuidos formados por clientes, servidores, servicios, bases de datos, aplicaciones móviles y recursos alojados en la nube. Dentro de esta evolución, la arquitectura cliente-servidor continúa siendo uno de los fundamentos más importantes, ya que permite dividir las responsabilidades entre un cliente que solicita información o servicios y un servidor encargado de procesar las solicitudes y proporcionar una respuesta. IBM señala que los sistemas cliente-servidor son una forma de sistema distribuido en la que los clientes dependen de uno o más servidores que administran recursos y servicios.

En las aplicaciones actuales, sin embargo, el concepto cliente-servidor ya no se limita a una computadora conectada directamente a un servidor. Una aplicación moderna puede estar formada por un frontend web, una aplicación móvil, una API, diversos microservicios, bases de datos y servicios externos que se comunican mediante redes. En estos escenarios intervienen tecnologías como DNS, TCP/IP, HTTP, HTTPS y TLS, además de formas de comunicación como REST, gRPC, WebSocket, Server-Sent Events, webhooks y sistemas de mensajería.

También se han desarrollado diferentes alternativas para organizar una aplicación, como las arquitecturas de dos, tres y N capas, los monolitos, los microservicios, la arquitectura orientada a eventos y la computación serverless. La selección entre estas opciones no debe realizarse únicamente porque una tecnología sea moderna, sino tomando en cuenta aspectos como la escalabilidad, disponibilidad, seguridad, consistencia de la información, costos y complejidad operacional. Los marcos de arquitectura actuales recomiendan encontrar un equilibrio entre confiabilidad, rendimiento, costos y facilidad de operación.

Finalmente, dentro de las prácticas contemporáneas surge el enfoque API-first, en el cual la interfaz de comunicación se diseña antes de comenzar la implementación completa del sistema. Esto permite establecer un contrato claro entre los diferentes componentes y facilita que distintos equipos trabajen simultáneamente. IBM define API-first como un enfoque donde las API constituyen la base de la aplicación y su contrato se especifica antes de desarrollar el código que las implementará.

1.1. Evolución del modelo cliente-servidor

Proceso cliente

El cliente es el componente encargado de iniciar una comunicación solicitando información o una operación. Puede tratarse de un navegador web, una aplicación de escritorio, una aplicación móvil o incluso otro servidor.

Por ejemplo, cuando una persona entra a una página web y presiona el botón "Iniciar sesión", el navegador funciona como cliente. Este genera una solicitud que contiene los datos necesarios y la envía al servidor encargado de autenticar al usuario.

Una característica importante es que el cliente generalmente se concentra en la interacción con el usuario y en presentar la información obtenida desde el servidor.

Proceso servidor

El servidor es el componente que recibe una solicitud, realiza algún tipo de procesamiento y devuelve una respuesta. El procesamiento puede involucrar validaciones, operaciones matemáticas, consultas a una base de datos, autenticación o comunicación con otros servicios.

IBM explica que en una arquitectura cliente-servidor el servidor administra recursos compartidos y los clientes normalmente interactúan con dichos recursos mediante un patrón de solicitud y respuesta.

Un ejemplo básico sería:

Cliente → solicitud → servidor → procesamiento → respuesta → cliente

En una tienda en línea, por ejemplo, el navegador puede solicitar la información de un producto. El servidor consulta la base de datos y posteriormente envía el nombre, precio, descripción y disponibilidad del producto.

Servicio

Un servicio representa una funcionalidad que un sistema pone a disposición de otros componentes. Por ejemplo, una aplicación puede tener un servicio de autenticación, uno de pagos, uno de correo electrónico y otro encargado de administrar usuarios.

La principal diferencia entre hablar simplemente de un servidor y hablar de un servicio es que el servidor hace referencia principalmente al componente que ejecuta los procesos, mientras que el servicio representa la funcionalidad que se proporciona.

Middleware

El middleware es software que actúa como intermediario entre aplicaciones, servicios o componentes. Su finalidad es facilitar su comunicación y evitar que cada sistema tenga que implementar todos los mecanismos de integración desde cero.

IBM describe el middleware como un intermediario situado entre aplicaciones que proporciona funciones o servicios para facilitar la comunicación entre ellas.

El middleware puede intervenir en tareas como autenticación, transformación de datos, comunicación entre servicios, administración de mensajes, registro de actividades y manejo de errores.

Sistema distribuido

Un sistema distribuido está formado por múltiples computadoras o procesos conectados mediante una red que colaboran para proporcionar una funcionalidad.

Desde la perspectiva del usuario, todos los componentes pueden parecer parte de un mismo sistema, aunque realmente el procesamiento se encuentre distribuido entre diferentes equipos.

Una aplicación de comercio electrónico, por ejemplo, podría disponer de un servidor web, otro servidor para las API, una base de datos, un servicio de pagos y un sistema de notificaciones. Aunque cada componente puede estar físicamente separado, todos colaboran para proporcionar el servicio al usuario.

El modelo cliente-servidor constituye precisamente una de las formas más comunes de sistema distribuido.

1.2. Fundamentos de red aplicados

Para comprender correctamente una arquitectura cliente-servidor es necesario conocer cómo viaja una solicitud desde un cliente hasta un servidor.

DNS

El DNS, Domain Name System, permite convertir nombres fáciles de recordar en direcciones IP.

Por ejemplo:

www.ejemplo.com → dirección IP del servidor

Sin DNS, una persona tendría que conocer directamente la dirección IP del servidor al que quiere conectarse. Cloudflare describe DNS como el sistema encargado de traducir los nombres de dominio a direcciones IP que las computadoras utilizan para localizar recursos en Internet.

Dirección IP

Una dirección IP permite identificar y localizar dispositivos dentro de una red.

IP se encarga principalmente del direccionamiento y enrutamiento de los paquetes para que puedan llegar desde el dispositivo de origen hasta su destino.

Por ejemplo, una dirección IPv4 puede escribirse de la siguiente manera:

192.168.1.25

Mientras que IPv6 utiliza direcciones considerablemente más largas.

Puertos

Una misma computadora puede ejecutar varios servicios simultáneamente. Los puertos permiten identificar a qué aplicación o servicio debe entregarse una comunicación.

Por ejemplo:

Servicio Puerto habitual

HTTP 80

HTTPS 443

DNS 53

MDN señala que un puerto representa un destino de comunicación dentro de un equipo conectado a una red y que HTTP utiliza normalmente el puerto 80, mientras que HTTPS utiliza el puerto 443.

TCP

TCP, Transmission Control Protocol, es un protocolo de transporte orientado a conexión diseñado para proporcionar una comunicación confiable.

TCP controla aspectos como el orden de los paquetes, confirmación de recepción y retransmisión cuando determinados datos no llegan correctamente.

Por ello se utiliza ampliamente en aplicaciones donde la información debe recibirse correctamente y en el orden adecuado.

HTTP

HTTP, Hypertext Transfer Protocol, es uno de los principales protocolos utilizados para la comunicación entre clientes y servidores web.

HTTP funciona principalmente mediante un modelo de petición y respuesta.

Un cliente puede solicitar:

GET /productos

El servidor procesa la solicitud y podría devolver:

200 OK

junto con los datos solicitados.

MDN define HTTP como un protocolo de la capa de aplicación basado en el modelo cliente-servidor, donde un cliente realiza una petición y recibe posteriormente una respuesta.

HTTPS

HTTPS representa HTTP protegido mediante mecanismos criptográficos proporcionados por TLS.

Esto permite que la información intercambiada entre cliente y servidor viaje cifrada, dificultando que un tercero pueda leer o modificar los datos durante su transmisión.

El uso de HTTPS resulta especialmente importante para contraseñas, información personal, datos empresariales o transacciones financieras.

Petición

Una petición o request es el mensaje que un cliente envía al servidor.

Puede incluir información como el recurso solicitado, método HTTP, encabezados, parámetros y datos enviados por el usuario.

Por ejemplo:

POST /usuarios

podría utilizarse para solicitar al servidor la creación de un nuevo usuario.

Respuesta

La respuesta o response es el mensaje generado por el servidor después de procesar la petición.

Normalmente contiene un código de estado y, cuando es necesario, información adicional.

Ejemplos comunes son 200 cuando una operación fue exitosa, 201 cuando se creó un recurso, 404 cuando no se encontró y 500 cuando ocurrió un error interno.

Latencia

La latencia representa el tiempo necesario para que la información viaje entre dos puntos de la red y sea procesada.

Cuanto mayor sea la distancia entre cliente y servidor, la cantidad de intermediarios o la congestión de la red, mayor puede ser la latencia.

En sistemas interactivos una latencia elevada provoca que el usuario perciba la aplicación como lenta, incluso cuando el servidor tenga suficiente capacidad de procesamiento.

TLS

TLS, Transport Layer Security, es el protocolo de seguridad utilizado ampliamente para proteger información transmitida a través de Internet.

Su objetivo es proporcionar confidencialidad e integridad durante la comunicación. Microsoft explica que TLS se sitúa entre las aplicaciones y TCP/IP y permite proteger los datos intercambiados entre cliente y servidor.

Cloudflare también señala que TLS es uno de los mecanismos fundamentales empleados para cifrar la comunicación entre navegadores y servidores web.

De manera simplificada, al acceder a una aplicación mediante HTTPS ocurre un proceso similar a:

DNS → IP del servidor → conexión → TLS → petición HTTP → procesamiento del servidor → respuesta

Cloudflare describe precisamente un flujo semejante al explicar la carga de una página web: primero se realiza DNS, posteriormente una conexión, se establece TLS y después se intercambian las solicitudes y respuestas HTTP.

1.3. Arquitecturas de software contemporáneas

Arquitectura de dos capas

En una arquitectura de dos capas, normalmente existen:

Cliente ↔ servidor o base de datos

Una parte considerable de la lógica puede encontrarse directamente en el cliente, mientras que el servidor administra principalmente los datos.

Es relativamente sencilla, pero puede generar un fuerte acoplamiento entre el cliente y la estructura de la base de datos. IBM identifica este modelo como una de las formas básicas de arquitectura cliente-servidor.

Arquitectura de tres capas

La arquitectura de tres capas separa normalmente:

Presentación → lógica de negocio → datos

La capa de presentación contiene la interfaz del usuario. La capa de lógica de negocio procesa las reglas de funcionamiento del sistema. La capa de datos administra la persistencia.

Esta separación facilita el mantenimiento porque la interfaz no necesita comunicarse directamente con la base de datos.

Arquitectura de N capas

Una arquitectura de N capas amplía esta idea creando capas adicionales dependiendo de las necesidades del sistema.

Podrían existir capas de presentación, API, servicios, acceso a datos, persistencia e integración con sistemas externos.

Red Hat indica que las arquitecturas N-tier organizan las aplicaciones mediante varias capas donde cada una cumple responsabilidades particulares.

Arquitectura monolítica

En una arquitectura monolítica, las funcionalidades principales de una aplicación se encuentran dentro de una única unidad de implementación.

Su ventaja principal es la simplicidad inicial. Resulta fácil desarrollar, probar y desplegar aplicaciones pequeñas.

Sin embargo, cuando un monolito crece demasiado pueden aparecer problemas de mantenimiento, acoplamiento y dificultad para actualizar determinadas partes sin volver a desplegar el sistema completo.

Monolito modular

El monolito modular busca mantener la facilidad operacional de un monolito, pero organizando internamente el software mediante módulos claramente separados.

Por ejemplo:

Usuarios | Reservaciones | Laboratorios | Notificaciones

Todos podrían formar parte de la misma aplicación y desplegarse juntos, pero cada módulo mantiene responsabilidades y límites bien definidos.

Este enfoque puede ser especialmente conveniente para aplicaciones pequeñas o medianas que requieren una buena organización, pero que todavía no justifican la complejidad de administrar numerosos microservicios.

Microservicios

Una arquitectura de microservicios divide una aplicación en servicios pequeños relativamente independientes.

Por ejemplo:

- Servicio de usuarios
- Servicio de pagos
- Servicio de inventario
- Servicio de notificaciones

Cada servicio puede desarrollarse y desplegarse independientemente.

Microsoft define los microservicios como componentes pequeños, autónomos y de bajo acoplamiento que se comunican a través de interfaces bien definidas.

Red Hat señala que permiten escalar y modificar servicios individuales sin necesidad de desplegar nuevamente toda la aplicación. [Red Hat](#)

Su principal desventaja es el incremento en la complejidad: aparecen problemas de comunicación por red, descubrimiento de servicios, observabilidad, seguridad, consistencia de datos y despliegues distribuidos.

Arquitectura orientada a eventos

En la arquitectura orientada a eventos, los componentes reaccionan ante acontecimientos que ocurren dentro del sistema.

Por ejemplo:

Pedido creado → evento → servicio de inventario

El mismo evento podría ser consumido también por facturación y notificaciones.

Este patrón ayuda a reducir el acoplamiento entre servicios, porque el productor del evento no necesariamente necesita conocer directamente todos los componentes que reaccionarán a él. AWS señala que este modelo permite construir sistemas en los que diferentes componentes intercambian información mediante eventos y pueden comunicarse de forma asíncrona.

Serverless

Serverless es un modelo en el cual el proveedor de nube administra gran parte de la infraestructura necesaria para ejecutar una aplicación.

No significa literalmente que no existan servidores. Significa que los desarrolladores no tienen que administrarlos directamente.

AWS señala que el proveedor puede encargarse del aprovisionamiento, escalado, mantenimiento, parches y gran parte de la infraestructura subyacente, permitiendo que los desarrolladores se concentren principalmente en las funcionalidades de la aplicación.

Una ventaja importante es la posibilidad de escalar automáticamente dependiendo de la demanda.

Backend as a Service — BaaS

BaaS proporciona funciones comunes de backend mediante servicios administrados y APIs.

Un proveedor BaaS puede ofrecer autenticación, bases de datos, almacenamiento de archivos o notificaciones sin que el desarrollador tenga que implementar toda la infraestructura desde cero.

AWS define BaaS como un modelo que proporciona funcionalidades de backend mediante APIs que pueden ser consumidas directamente por las aplicaciones.

1.4. Estilos y protocolos de comunicación

No existe un único protocolo apropiado para todas las aplicaciones. La elección depende de la velocidad requerida, cantidad de datos, compatibilidad, forma de comunicación y complejidad que pueda manejar el equipo.

Tecnología	Funcionamiento principal	Uso recomendado
REST	Petición y respuesta sobre HTTP	APIs web y móviles
RPC/gRPC	Invocación de procedimientos remotos	Comunicación entre servicios
WebSocket	Comunicación bidireccional persistente	Chats, videojuegos, sistemas en tiempo real
SSE	Servidor envía continuamente información al cliente	Notificaciones, paneles y actualizaciones
Webhook	Un sistema envía una petición cuando ocurre un evento	Integraciones entre plataformas
Mensajería	Intercambio asíncrono mediante colas o brokers	Procesamiento desacoplado y sistemas distribuidos

REST

REST, Representational State Transfer, es uno de los estilos más utilizados para desarrollar APIs web.

Normalmente emplea HTTP y operaciones como GET, POST, PUT, PATCH y DELETE.

Por ejemplo:

GET /alumnos/25

podría utilizarse para recuperar los datos del alumno número 25.

IBM señala que las APIs REST se caracterizan, entre otras cosas, por mantener separación entre cliente y servidor, no depender del estado de solicitudes anteriores y poder aprovechar mecanismos de caché.

REST resulta especialmente conveniente para aplicaciones web y móviles por su sencillez y amplia compatibilidad.

RPC y gRPC

RPC, Remote Procedure Call, permite solicitar la ejecución de procedimientos que se encuentran en otra computadora.

gRPC es una implementación moderna de este concepto orientada a comunicaciones de alto rendimiento.

Microsoft señala que gRPC abstrae aspectos de comunicación de red, serialización y ejecución remota, y utiliza HTTP/2 para proporcionar características como multiplexación y streaming bidireccional.

Además, normalmente utiliza archivos .proto para definir previamente los servicios y los mensajes que serán intercambiados.

Resulta particularmente útil en comunicaciones internas entre microservicios.

WebSocket

WebSocket mantiene una conexión persistente entre cliente y servidor y permite comunicación bidireccional.

Esto significa que tanto el cliente como el servidor pueden enviar información sin esperar a que el otro genere primero una nueva petición HTTP.

Es apropiado para chats, videojuegos, plataformas colaborativas, seguimiento en tiempo real y aplicaciones donde los datos cambian constantemente.

MDN explica que WebSocket crea una comunicación basada en una conexión persistente sobre TCP después de realizar el proceso inicial de negociación.

Server-Sent Events

Los **Server-Sent Events** o **SSE** permiten que un servidor mantenga una conexión abierta y envíe información continuamente al cliente.

A diferencia de WebSocket, se orienta principalmente a comunicación:

Servidor → cliente

MDN explica que mediante SSE una página puede recibir nuevos datos enviados por el servidor sin tener que solicitar continuamente nuevas actualizaciones.

Es útil para noticias en tiempo real, seguimiento de procesos, notificaciones y paneles de monitoreo.

Webhooks

Un **webhook** permite que un servicio notifique automáticamente a otro cuando ocurre un determinado acontecimiento.

Por ejemplo:

Pago aprobado → webhook → sistema de tienda

En lugar de que la tienda consulte cada poco segundo si el pago fue aprobado, el proveedor de pagos envía automáticamente una petición HTTP cuando cambia su estado.

Microsoft describe un webhook como un mecanismo basado en HTTP donde un sistema envía notificaciones a un punto de conexión cuando se produce determinado evento.

Mensajería

Los sistemas de **mensajería** permiten que diferentes aplicaciones intercambien mensajes mediante tecnologías intermedias, normalmente colas o brokers.

La diferencia frente a una comunicación síncrona es que el productor puede publicar un mensaje sin necesidad de que el consumidor se encuentre procesándolo exactamente en ese mismo momento.

Esto permite desacoplar componentes y facilita la creación de procesos asíncronos.

Por ejemplo:

Usuario realiza compra → cola → servicio de correo

El proceso de compra puede terminar mientras el servicio de correo procesa posteriormente el envío de la confirmación.

1.5. Separación de responsabilidades y criterios arquitectónicos

Una arquitectura no debe seleccionarse únicamente por popularidad. Debe responder al problema que intenta resolver.

Separación de responsabilidades

La **separación de responsabilidades** busca que cada parte del sistema tenga funciones claramente definidas.

Por ejemplo, el frontend debería concentrarse principalmente en la interfaz, mientras que el backend procesa reglas de negocio y la base de datos conserva la información.

Una separación adecuada facilita las pruebas, el mantenimiento y la modificación del sistema.

Escalabilidad

La **escalabilidad** representa la capacidad de un sistema para soportar un crecimiento en usuarios, solicitudes o información.

El escalado puede realizarse verticalmente, utilizando servidores con mayores recursos, u horizontalmente, agregando varias instancias.

Los sistemas distribuidos y servicios en la nube pueden escalar determinadas partes independientemente, aunque esta capacidad normalmente introduce costos y complejidad adicionales. Microsoft recomienda analizar el crecimiento de la demanda junto con su impacto económico.

Disponibilidad

La **disponibilidad** representa la capacidad del sistema para permanecer accesible.

Para mejorarla pueden utilizarse varias instancias del servicio, balanceadores, redundancia de datos y diferentes zonas o regiones.

Microsoft señala que distribuir recursos entre diferentes zonas permite reducir el impacto producido por la pérdida de un centro de datos.

Consistencia

La **consistencia** se relaciona con que diferentes componentes dispongan de información coherente.

En una aplicación centralizada puede resultar relativamente sencillo mantener una única base de datos.

En sistemas altamente distribuidos, diferentes servicios pueden tener su propia información, haciendo más complejo mantener todos los datos sincronizados inmediatamente.

Por esta razón, la distribución proporciona ventajas de escalabilidad y autonomía, pero también obliga a definir mecanismos de sincronización y tratamiento de errores.

Costo

Una arquitectura más compleja no necesariamente es mejor.

Más servidores, bases de datos, herramientas de observabilidad, balanceadores y recursos de nube pueden incrementar considerablemente el costo.

Por esta razón, la arquitectura debe ser proporcional al tamaño y necesidades reales del sistema.

Microsoft destaca que determinados diseños destinados a mejorar la disponibilidad o rendimiento pueden aumentar el gasto y requieren evaluar cuidadosamente las compensaciones entre costo y beneficio.

Complejidad

A medida que aumenta la cantidad de componentes, también aumenta la dificultad para administrarlos.

Una aplicación con veinte microservicios requiere mecanismos para desplegar, monitorear, actualizar y diagnosticar cada uno de ellos.

Por este motivo, para determinados proyectos un monolito modular puede resultar más conveniente que comenzar directamente con microservicios.

Deuda operativa

La **deuda operativa** aparece cuando decisiones técnicas incrementan progresivamente el esfuerzo necesario para administrar un sistema.

Puede manifestarse en configuraciones manuales, documentación insuficiente, servidores difíciles de actualizar, servicios duplicados, despliegues complicados o falta de monitoreo.

Una arquitectura que inicialmente parece muy avanzada puede terminar generando deuda operativa si el equipo no dispone de tiempo, herramientas o conocimientos suficientes para mantenerla.

Por lo tanto, una decisión arquitectónica adecuada debe equilibrar:

escalabilidad + disponibilidad + consistencia + costo + complejidad + capacidad real del equipo.

1.6. Diseño API-first, diagramas C4, contrato inicial y ADR

Diseño API-first

En el enfoque tradicional code-first, los desarrolladores pueden comenzar creando el backend y posteriormente definir cómo será utilizada su API.

API-first propone invertir el proceso:

Primero se diseña la API → después se desarrolla su implementación.

IBM señala que API-first trata a la API como la base de la aplicación y establece primero su contrato, frecuentemente mediante una especificación como OpenAPI.

Por ejemplo, antes de programar un sistema de reservación podría determinarse que existe:

POST /reservaciones

Entrada:

```
{  
  "usuario_id": 25,  
  "laboratorio_id": 4,  
  "fecha": "2026-10-10",  
  "hora_inicio": "12:00"  
}
```

Respuesta:

```
{  
  "id": 302,  
  "estado": "confirmada"  
}
```

El frontend puede comenzar a trabajar utilizando este contrato, aunque el backend todavía no esté completamente terminado.

Esto permite que ambos equipos desarrollen partes diferentes del sistema de manera paralela.

Contrato inicial

El contrato de una API define cómo deben comunicarse cliente y servidor.

Debe establecer aspectos como rutas disponibles, métodos HTTP, información que recibe la API, estructura de las respuestas, códigos de estado y posibles errores.

Red Hat señala que las APIs pueden considerarse contratos que establecen la forma en que una aplicación debe realizar una solicitud y cómo responderá el otro sistema.

Con una especificación como OpenAPI, este contrato puede convertirse posteriormente en documentación interactiva mediante herramientas como Swagger.

Diagramas C4 simplificados

El modelo C4 permite representar visualmente la arquitectura de un sistema mediante diferentes niveles de detalle.

Para trabajos académicos generalmente son especialmente útiles los primeros niveles.

El diagrama de contexto C1 muestra el sistema completo, los usuarios y los sistemas externos con los que se comunica.

Un ejemplo simplificado sería:

Estudiante → Sistema de reservación → Servicio de correo

El diagrama de contenedores C2 muestra los principales componentes técnicos que existen dentro del sistema.

Por ejemplo:

Usuario → aplicación web → API backend → base de datos

y:

API backend → servicio de correo

De esta manera, C1 ayuda a comprender con quién se relaciona el sistema, mientras que C2 ayuda a comprender de qué partes principales está formado.

Registros de decisión arquitectónica — ADR

Un ADR, Architectural Decision Record, es un documento breve que registra una decisión importante tomada durante el diseño de la arquitectura.

AWS define un ADR como un documento que registra una decisión arquitectónica, su contexto y sus consecuencias. [Documentación de AWS](#)

Por ejemplo:

ADR-001: Selección de FastAPI para desarrollar la API

Contexto: Se requiere desarrollar rápidamente una API REST utilizando Python.

Opciones consideradas: FastAPI, Flask y Django REST Framework.

Decisión: Utilizar FastAPI.

Consecuencias: Se obtiene validación automática, generación de documentación y buen rendimiento, pero el equipo necesitará conocimientos de Python y del ecosistema FastAPI.

Los ADR ayudan a evitar que meses después el equipo se pregunte por qué se seleccionó determinada tecnología.

Microsoft indica que un ADR debería conservar el contexto, las alternativas consideradas, la decisión, sus consecuencias y su estado.

La combinación de API-first + C4 + ADR proporciona una documentación muy completa:

C4 explica cómo está organizado el sistema.

El contrato de la API explica cómo se comunican los componentes.

Los ADR explican por qué se tomaron determinadas decisiones.

Conclusión

Las arquitecturas cliente-servidor continúan siendo uno de los fundamentos esenciales para el desarrollo de sistemas modernos. Aunque originalmente el modelo consistía principalmente en un cliente que realizaba peticiones a un servidor central, actualmente las aplicaciones pueden estar distribuidas entre numerosos componentes, incluyendo servicios web, bases de datos, aplicaciones móviles, proveedores externos y plataformas de nube.

Comprender elementos como DNS, direcciones IP, puertos, TCP, HTTP, HTTPS y TLS permite entender el proceso que se produce desde que un usuario solicita información hasta que obtiene la respuesta del servidor. Estos fundamentos también resultan indispensables al desarrollar APIs, debido a que la mayoría de las comunicaciones modernas dependen de tecnologías de red y protocolos estandarizados.

Asimismo, existen diferentes estilos arquitectónicos que responden a necesidades distintas. Las arquitecturas de capas permiten separar responsabilidades; los monolitos proporcionan simplicidad; los monolitos modulares mejoran la organización sin incrementar excesivamente la complejidad operacional; los microservicios ofrecen independencia y escalabilidad; las arquitecturas orientadas a eventos permiten desacoplar componentes; y las soluciones serverless y BaaS reducen parte de las responsabilidades relacionadas con la administración de infraestructura. No existe una arquitectura universalmente superior. La decisión depende de las características del proyecto, los recursos disponibles y los objetivos que se buscan alcanzar.

De la misma manera, REST, gRPC, WebSocket, Server-Sent Events, webhooks y la mensajería solucionan problemas de comunicación diferentes. REST resulta adecuado para gran cantidad de APIs web; gRPC puede utilizarse para comunicaciones eficientes entre servicios; WebSocket resulta conveniente para interacción bidireccional en tiempo real; SSE facilita el envío continuo de información desde el servidor; los webhooks permiten reaccionar ante eventos externos y los sistemas de mensajería posibilitan crear procesos asíncronos y desacoplados.

Finalmente, el enfoque API-first representa una práctica importante en los sistemas contemporáneos porque permite establecer primero las reglas de comunicación y después desarrollar los componentes que las implementarán. Complementar este enfoque con diagramas C4 y registros ADR permite representar la arquitectura, definir contratos claros y mantener evidencia de las razones que originaron las decisiones técnicas. En conjunto, estas prácticas ayudan a desarrollar sistemas más mantenibles, comprensibles, escalables y preparados para evolucionar con las necesidades de sus usuarios.

Fuentes de consulta — formato APA 7

- IBM. (s. f.). *¿Qué son los sistemas distribuidos?* IBM. <https://www.ibm.com/es-es/think/topics/distributed-systems>
- IBM. (s. f.). *¿Qué es una API (interfaz de programación de aplicaciones)?* IBM. <https://www.ibm.com/es-es/think/topics/api>
- Cloudflare. (s. f.). *¿Qué es DNS? ¿Cómo funciona el DNS?* Cloudflare Learning Center. <https://www.cloudflare.com/es-es/learning/dns/what-is-dns/>
- MDN Web Docs. (s. f.). *Generalidades del protocolo HTTP*. Mozilla. <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
- Microsoft. (s. f.). *Estilo de arquitectura de microservicios*. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/microservices>
- Red Hat. (s. f.). *¿Qué es una arquitectura de aplicaciones?* Red Hat. <https://www.redhat.com/es/topics/cloud-native-apps/what-is-an-application-architecture>